



UNIVERSIDAD DEL BÍO-BÍO

UNIVERSIDAD DEL BÍO-BÍO
Facultad de Ciencias Empresariales

MANUAL DE USUARIO Y TUTORIAL CIM DEFINITIVO v6.0 Sistema de Manufactura Integrada por Computador

Autor:	Leonardo Araya Labarca
Carrera:	Ing. de Ejecución en Computación e Informática
Versión:	v6.0 - Pre-hardware 100% automatizable
Fecha:	28 de julio de 2026
Repositorio:	haloharry973/CIM-DEFINITIVO

Este manual guía la instalación, configuración y operación segura del sistema CIM en modo simulación y prepara el paso a laboratorio con protocolo de seguridad.

ÍNDICE MANUAL / TUTORIAL

1. Introducción al Sistema CIM	3
2. Requisitos e Instalación	4
3. Arquitectura y Flujo Operativo	6
4. Tutorial Paso a Paso – Instalación Android	8
5. Tutorial Firmware ESP32 / Wemos D1 R32	12
6. Uso del Sistema – Coordinador, PLC, Manufactura, Calidad, Almacén, Wear	14
7. Comunicación, Identidad y Seguridad	18
8. Simulación y Herramientas de Visión	19
9. Solución de Problemas Frecuentes	20
10. Protocolo de Laboratorio y Seguridad	22

1. INTRODUCCIÓN AL SISTEMA CIM

Bienvenido al **Manual de Usuario y Tutorial del Sistema CIM DEFINITIVO v6.0**. Este documento te guiará desde cero para instalar, configurar y operar la celda de manufactura integrada por computador desarrollada en la Universidad del Bío-Bío. Al finalizar podrás: instalar las 6 APKs, flashear los ESP32, iniciar el hub Coordinador, admitir estaciones y ejecutar un ciclo completo de pallet en simulación segura sin energizar actuadores peligrosos.

¿Qué es CIM DEFINITIVO?

Es una refactorización completa de la celda CIM del laboratorio: en lugar de PLCs heterogéneos, usa tablets/celulares Android como controladores de estación y placas Wemos D1 R32 (ESP32) como interfaz de campo BLE. La lógica de negocio (máquina de estados pallet, admisión por UUID, visión conservadora) vive en Kotlin y es validada automáticamente por CI. El estado actual es pre-hardware 100% automatizable: puedes probar todo el software sin riesgo.

Público objetivo:

- Estudiantes IECI en Práctica II / Tesis
- Encargado de laboratorio
- Futuros mantenedores del código

2. REQUISITOS E INSTALACIÓN

2.1 Requisitos de Software

Recurso	Versión	Uso
Git	2.x +	Obtener código
Python 3	3.11+	Validadores
JDK	17	Gradle/Android
Android SDK	compileSdk 35, buildToolsVersion 35	Compilar apps
Android Studio o cmdline-tools	Android 8.0+	ADB, emulador
Chrome/Edge	Reciente	Generar PDFs (opcional)
Arduino IDE / CLI / PlatformIO	1.8.02.x	Flasheo ESP32

2.2 Requisitos Hardware (para paso a laboratorio, NO para este tutorial)

- Wemos D1 R32 (4 unidades mínimo) + cables USB
- Fuente 12V, fusible, tablero con relé GPIO5 y sensor GPIO34 con interfaz protegida
- Tablet/celular Android 8.0+ (6 dispositivos ideal)
- Red Wi-Fi controlada, sin acceso internet obligatorio
- E-Stop físico independiente probado – OBLIGATORIO antes de cualquier actuador
- Zona delimitada sin personas en radio movimiento Scorbob

NOTA DE SEGURIDAD:

Este tutorial cubre solo instalación y simulación. Nunca conectes actuadores reales sin E-Stop físico que corte energía de actuadores independientemente del software, interlocks, supervisor y acta firmada. Ver MANUAL_OPERATIVO_LABORATORIO.md.

3. ARQUITECTURA Y FLUJO OPERATIVO

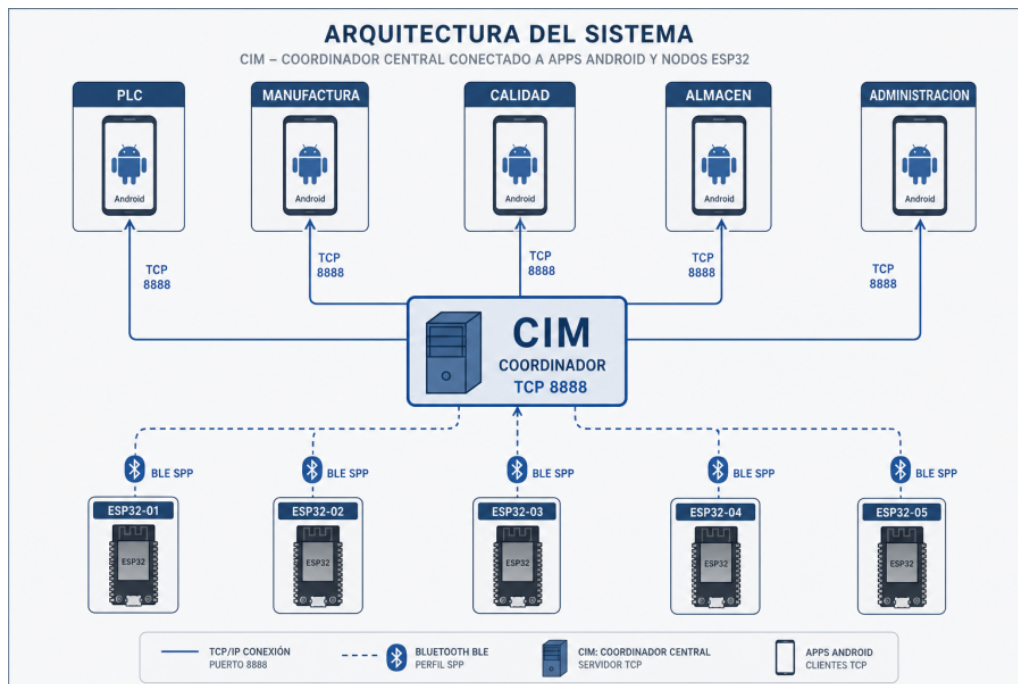


Figura: Arquitectura 3 capas CIM v6.0

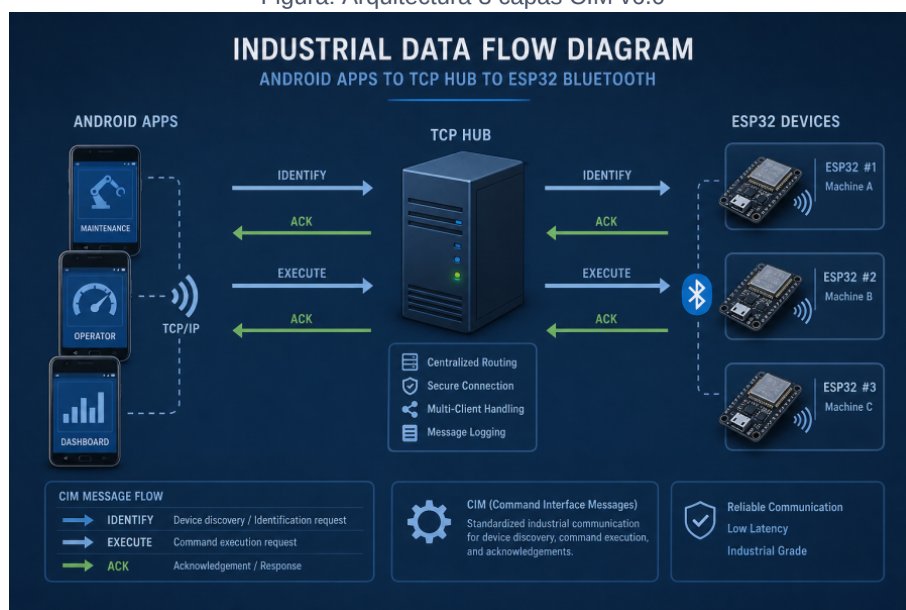


Figura: Flujo de pallets y estados (IDLE, MOVING, PROCESSING, QUALITY, STORAGE, BLOCKED)

Flujo simplificado de un pallet:

- 1. PLC detecta pallet (sensor GPIO34) -> envía PALLET_ARRIVED al Coordinador.
- 2. Coordinador autoriza y ordena a Manufactura.
- 3. Manufactura ejecuta G-code robot + láser (en simulación solo telemetría).
- 4. Manufactura marca COMPLETED -> Coordinador envía a Calidad.
- 5. Calidad captura cámara, detecta ArUco, YOLO bestMH.pt decide PASS/FAIL/REVIEW_REQUIRED.

-
- 6. Según decisión, Coordinador deriva a Almacén (PASS) o a revisión (FAIL/BLOCKED).
 - 7. Almacén guarda en rack y confirma STORED.

4. TUTORIAL PASO A PASO – INSTALACIÓN ANDROID

Paso 0 – Clonar y validar repositorio

Abre terminal en carpeta deseada:

```
git clone https://github.com/haloharry973/CIM-DEFINITIVO.git
cd CIM-DEFINITIVO
python3 tools/validate_firmware_contract.py --quiet
python3 tools/validate_system_100.py --quiet
python3 tools/prehardware_readiness.py --quiet
python3 -m compileall -q tools
git diff --check
```

Debes ver PASS / sin errores. Si falla, corrige el archivo indicado.

Paso 1 – Preparar SDK

Instala JDK 17 (no 21). Configura ANDROID_HOME / ANDROID_SDK_ROOT apuntando a tu SDK. Acepta licencias: sdkmanager --licenses. Asegura platform-tools (ADB) en PATH.

Paso 2 – Compilar APKs

En Linux/Mac:

```
cd config
./gradlew testAllModules
./gradlew lintAll
./gradlew buildAllApks validateApks writeApkChecksums
```

En Windows PowerShell:

```
cd config
.\gradlew.bat testAllModules
.\gradlew.bat lintAll
.\gradlew.bat buildAllApks validateApks writeApkChecksums
```

Las APK debug quedan en config/output-apks/ con SHA256SUMS.txt. No las subas al repo (están en .gitignore). Si no tienes SDK local, revisa la pestaña Actions del repo: descarga artefactos del último CI verde.

Paso 3 – Instalar en dispositivos

Conecta tablet por USB, habilita depuración USB y:

```
adb devices # debe listar dispositivo
.\tools\powershell\Instalar-APKs.ps1 # en Windows
# o manual: adb install -r config/output-apks/app-coordinador-debug.apk
adb install -r config/output-apks/app-plc-debug.apk
adb install -r config/output-apks/app-manufactura-debug.apk
adb install -r config/output-apks/app-calidad-debug.apk
adb install -r config/output-apks/app-almacen-debug.apk
adb install -r config/output-apks/wear-coordinador-debug.apk # si tienes Wear
```

Paso 4 – Primer arranque (orden importante)

IMPORTANTE: Abre primero **Coordinador**. Pulsa 'Iniciar Hub'. Anota IP y puerto mostrados. Luego abre PLC, Manufactura, Calidad, Almacén en ese orden. Cada estación pedirá admisión. En Coordinador verás solicitudes con MAC, UUID, versión, capacidades. Acepta si UUID coincide con lista esperada. Los rechazados quedan bloqueados persistentemente (puedes desbloquear desde Coordinador).

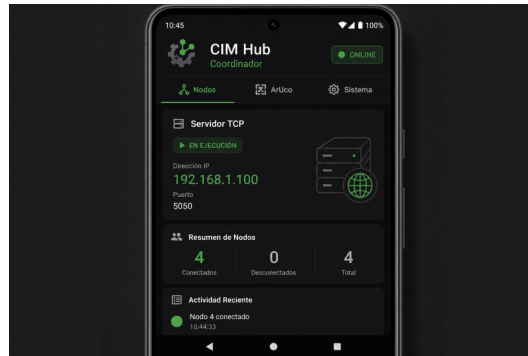


Figura: UI App Coordinador – Hub e identidad

5. TUTORIAL FIRMWARE ESP32 / WEMOS D1 R32

Paso 5 – Revisar firmware canónico

Solo usar esp32/firmware/ (no archive/):

```
esp32/firmware/esp32_plc_master.ino esp32/firmware/esp32_scorbot_manufactura.ino  
esp32/firmware/esp32_scorbot_calidad.ino esp32/firmware/esp32_scorbot_almacen.ino  
esp32/firmware/cim_ble_firmware.h
```

Paso 6 – Flasheo

Con Arduino IDE: selecciona placa 'WEMOS D1 R32', puerto COMx, abre .ino y Subir. Con CLI:

```
.\tools\powershell\Flashear-ESP32.ps1 -Port COM3 -Firmware PLC # o arduino-cli compile  
--fqbn esp32:esp32:d1_mini32 esp32/firmware/esp32_plc_master.ino arduino-cli upload -p  
COM3 --fqbn esp32:esp32:d1_mini32 esp32/firmware/esp32_plc_master.ino
```

Paso 7 – Captura CIM_ID

Abre Monitor Serie 115200 baud. Debes ver anuncio tipo CIM_ID:PLC_001:UUID:CAP:... Anota y registra en bitácora con fecha, operador, commit, placa, puerto.

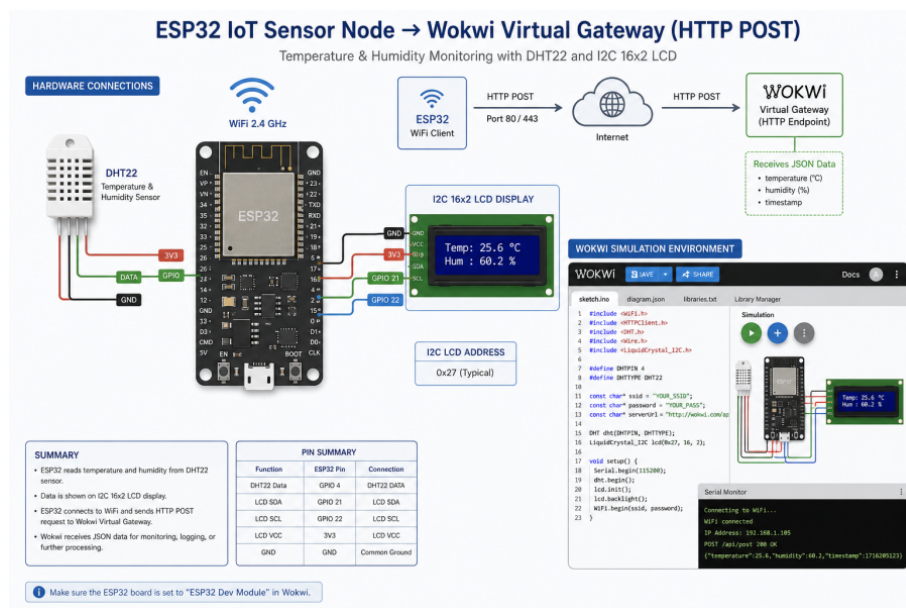
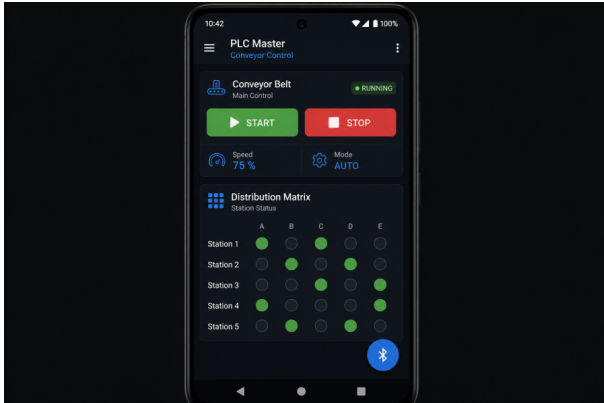


Figura: Simulación Wokwi ESP32 – referencia para captura CIM_ID

6. USO DEL SISTEMA – ESTACIONES

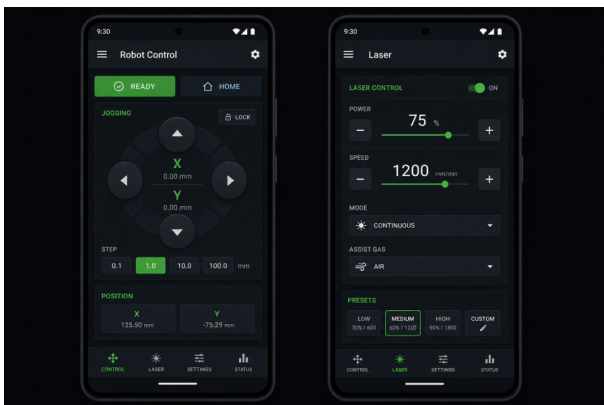
6.1 Estación PLC / Cinta

Gestiona cinta transportadora y sensor de proximidad. Eventos: PALLET_ARRIVED, PALLET_LEFT. Muestra estado sensor GPIO34 (simulado sin hardware). Botones: Iniciar cinta (solo telemetría en pre-hardware), Detener, Enviar evento manual.



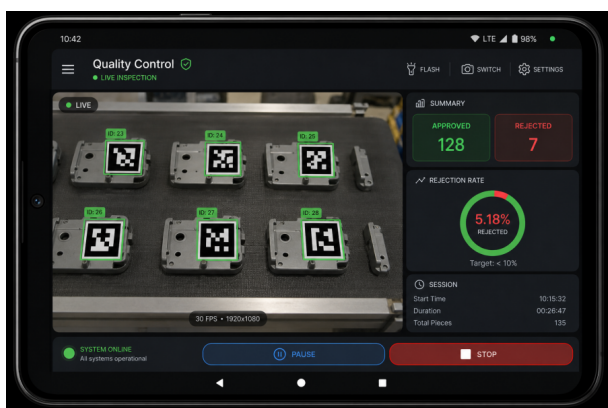
6.2 Estación Manufactura / Scorbot + Láser

Controla robot y láser. En pre-hardware no energiza. Permite cargar G-code, ver posiciones, ejecutar simulación de trayectoria. Orden láser solo con autorización. Botones: Home, Ejecutar G-code (sim), Parada segura.



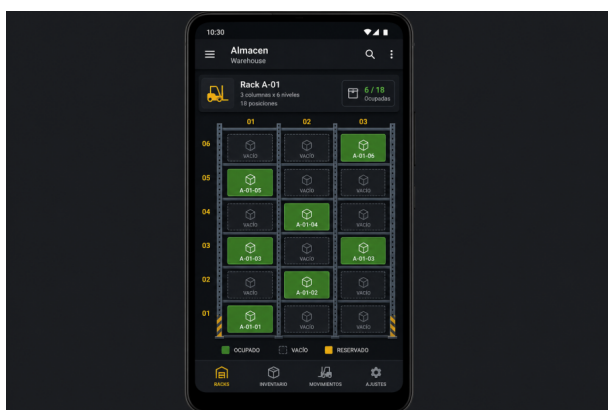
6.3 Estación Calidad / Cámara + ArUco + YOLO

Cámara con CameraX, detección ArUco, inferencia YOLO (TFLite pendiente). En simulación muestra resultado REVIEW_REQUIRED ante baja confianza. Botones: Iniciar cámara, Capturar, Detectar ArUco, Evaluar Calidad. Resultado PASS/FAIL asociado a pallet.



6.4 Estación Almacén / Rack

Gestiona posiciones de rack para guardar/recuperar. Operaciones trazables con pallet ID. Botones: Buscar posición libre, Guardar, Recuperar, Listar inventario.



6.5 App Wear – Supervisión

Vista compacta de estados para smartwatch. Solo supervisión, no control de seguridad. Muestra lista pallets, estados, alertas.

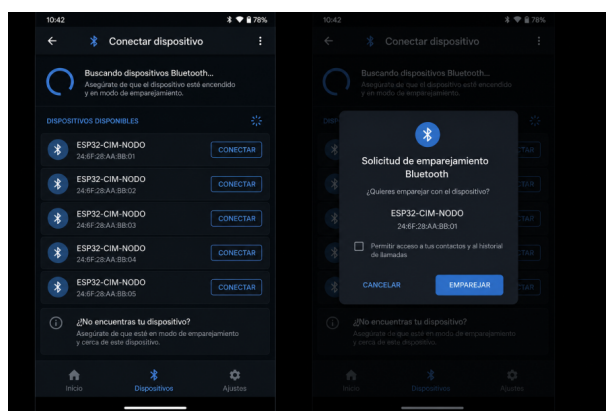


Figura: Conexión Bluetooth – Lista de dispositivos y estado BLE

7. COMUNICACIÓN, IDENTIDAD Y SEGURIDAD

Formato mensaje:

```
ID|TIMESTAMP|SOURCE_MAC|SOURCE_APP|DEST_MAC|DEST_APP|CMD|PRIORITY|SESSION|PAYLOAD
```

Identidad:

Cada estación tiene UUID canónico, MAC, tipo (PLC, MANUFACTURA, CALIDAD, ALMACEN, COORDINADOR), versión y capacidades. StationIdentityPolicy valida: UUID coincidente, tipo esperado, capacidades compatibles. Incompatible = bloqueo persistente. Token emparejamiento viaja como SHA-256, no texto plano.

Seguridad conservadora:

- Fragmentación BLE con MTU 20 bytes inicial, reensamblado seguro con timeout.
- Máquina pallet bloquea transiciones imposibles (ej: STORAGE -> PROCESSING sin pasar por QUALITY).
- Visión conservadora: baja confianza = REVIEW_REQUIRED, no PASS.
- Relé GPIO5 arranca en estado seguro (abierto).
- E-Stop independiente obligatorio – software nunca único control.

8. SIMULACIÓN Y HERRAMIENTAS DE VISIÓN

Simuladores sin hardware:

```
python3 tools/hub_simulator.py # simula hub coordinador y admisión python3
tools/vision_safety_simulator.py # 1000 casos seguridad visión python3
tools/inspect_yolo_checkpoint.py --checkpoint assets/models/bestMH.pt python3
tools/export_yolo_to_tflite.py --checkpoint bestMH.pt --output bestMH.tflite python3
tools/tflite_yolo_test.py
```

Estos comandos permiten validar lógica sin banco.

9. SOLUCIÓN DE PROBLEMAS FRECUENTES

Problema	Causa probable	Solución
APK no instala	Depuración USB desactivada / firma distinta	habilita depuración, desinstala versión previa, adb install -r
Coordinador no ve estación	IP/puerto incorrecto / firewall / red distinta	Verifica Wi-Fi misma red, revisa IP mostrada en Coordinador, desactiva firewall
BLE no conecta	Permisos ubicación / Bluetooth desactivado / no se da permiso de ubicación	Concede permisos de ubicación Bluetooth, revisa CIM_ID en Monitor Serie
validate_system_100 FAIL	Falta archivo activo o hay APK versionadas en log	desarrolla y corrige estructura, git rm APKs versionadas
Lint falla	Uso API deprecada / resource no encontrado	Ejecuta ./gradlew :app:lintDebug --stacktrace, corrige
Cámara negra en Calidad	Permiso cámara no concedido / emulador no tiene cámara	Concede permiso, prueba en dispositivo físico con cámara
E-Stop no corta energía	Cableado incorrecto / relé sin alimentación	DETENER TODO, revisar esquema eléctrico, no continuar sin E-Stop probado

10. PROTOCOLO DE LABORATORIO Y SEGURIDAD

Checklist antes de energizar (reproducido de `MANUAL_OPERATIVO_LABORATORIO.md`):

- Operador y supervisor identificados; área delimitada y sin personas en zona de movimiento.
- E-Stop físico independiente probado y accesible (corta energía actuadores, no solo orden software).
- Robot/láser sin permiso operación; relé GPIO5 sin carga durante pruebas iniciales.
- Fuente, fusible, masas, conectores y polaridad inspeccionados.
- GPIO34 con interfaz eléctrica definida (no aplicar tensión fuera rango ESP32 ni dejar flotante).
- Firmware tomado de esp32/firmware/, commit y puertos anotados.

Secuencia arranque controlado:

- 1. Energizar lógica con actuadores aislados.
- 2. Abrir monitor serie y confirmar anuncio CIM_ID esperado.
- 3. Conectar una estación BLE y comprobar UUID/tipo/capacidades.
- 4. Ejecutar solo comandos lectura/telemetría. Registrar timeout, desconexión o rechazo.
- 5. Probar sensor y relé sin carga, con una persona observando E-Stop.
- 6. Ante anomalía: accionar E-Stop, retirar energía actuadores, conservar logs, registrar incidencia.

Criterios de detención inmediata:

- Identidad distinta / CIM_ID inesperado
- Relé activo al arranque
- E-Stop ineficaz
- GPIO flotante / lecturas erráticas
- Pérdida enlace en estado riesgo
- Movimiento inesperado robot/cinta
- Ausencia supervisor

Documento fin – Tutorial CIM DEFINITIVO v6.0 – Universidad del Bío-Bío – Leonardo Araya 2026

Para evidencia y bitácora usar docs/deliverables/BITACORA_VALIDACION.md – Cada ensayo debe registrar fecha, operador, commit, APK/firmware, dispositivo, escenario, esperado, observado, logs/captura, incidencia, acción y aprobación.